

Interview Exercise

Vulnerability Snapshot Change Monitor

Backend service exercise for vulnerability assessment and SBOM workflows

Audience	Backend developers joining a product security services / vulnerability assessment team
Expected timebox	2 hours for core implementation, with README notes for unfinished items
Primary stack	Python, Flask, SQLAlchemy, Docker, relational database
Main signal	Deterministic diff logic, clean API boundaries, validation, persistence, tests, and judgment

Vulnerability Snapshot Change Monitor

Build an HTTP service, packaged as a Docker container, that ingests vulnerability snapshots for a product version, compares each accepted snapshot to the previous accepted snapshot for the same product version, stores the results, and exposes the change history through an API.

This simulates a product security workflow where updated vulnerability assessment data is received over time and must be converted into an auditable change history for SBOM-backed products.

Timebox and submission

- Expected time: 2 hours. Prioritize correctness, clarity, and tests over extra features.
- Submission: Git repository containing source code, Dockerfile, README, database setup script or migration, and tests.
- Required stack: Python 3.11 or newer, Flask, Docker, and a relational database. PostgreSQL is preferred. SQLite is acceptable for tests.
- It is acceptable to leave non-core items unfinished if the README clearly explains what remains and how you would complete it.

Core goal

The service must detect and persist these changes between snapshots:

- New vulnerabilities: finding exists in the current snapshot but not in the previous snapshot.
- Resolved vulnerabilities: finding existed in the previous snapshot but does not exist in the current snapshot.
- Severity changes: finding exists in both snapshots, but severity changed.
- Affected status changes: finding exists in both snapshots, but affected_status changed.
- Unchanged count: finding exists in both snapshots and neither severity nor affected_status changed. Unchanged findings do not need individual change records.

Out of scope for the core exercise

- Authentication and authorization.
- User interface.
- Distributed workers or queues.
- Full CVE enrichment, EPSS enrichment, exploit intelligence ingestion, or SBOM parsing.
- Complex out-of-order backfill support. See snapshot ordering rules below.

Configuration

Configure the service using environment variables. Document defaults and any additional values in the README.

Variable	Required	Notes
DB_HOST	Yes	Database host.
DB_PORT	Yes	Database port.
DB_NAME	Yes	Database name.
DB_USER	Yes	Database user.
DB_PASSWORD	Yes	Database password.
PORT	Yes	HTTP port. Default to 8080 if not supplied.
CONCURRENCY	No	Optional. Use a sensible default and document how it is used, or document that it is currently ignored.

Input model

A snapshot represents the vulnerability state of one product version at one point in time. All fields shown below are required except `package_url`.

```
{
  "product_name": "ExampleDevice",
  "product_version": "1.2.3",
  "source": "daily-vulnerability-scan",
  "snapshot_time": "2026-01-01T10:00:00Z",
  "findings": [
    {
      "vulnerability_id": "CVE-2024-12345",
      "component_name": "openssl",
      "component_version": "1.1.1k",
      "package_url": "pkg:generic/openssl@1.1.1k",
      "severity": "high",
      "cvss_score": 8.1,
      "affected_status": "affected"
    }
  ]
}
```

Allowed values

Field	Allowed values
severity	critical, high, medium, low, none, unknown
affected_status	affected, not_affected, fixed, under_investigation, accepted_risk
cvss_score	Number from 0.0 to 10.0 inclusive. Use one decimal when possible.
snapshot_time	RFC 3339 / ISO 8601 UTC timestamp, for example 2026-01-01T10:00:00Z.

Finding identity and matching key

Use this matching key to decide whether a finding in one snapshot is the same finding in another snapshot:

```
vulnerability_id + component_name + component_version
```

- The same vulnerability affecting two different components or component versions must be treated as two distinct findings.
- `package_url` should be stored when present, but it is not part of the required matching key for this exercise.
- A submitted snapshot must not contain duplicate findings with the same matching key. Reject the request with 400 Bad Request if duplicates are present.
- Normalize matching-key fields by trimming surrounding whitespace. Do not require case folding unless you document it.

Snapshot ordering and idempotency

For the core exercise, implement the following deterministic behavior:

- For each `product_name` and `product_version`, accepted snapshots must be submitted in strictly increasing `snapshot_time` order.
- The previous snapshot is the latest accepted snapshot for the same `product_name` and `product_version` with an earlier `snapshot_time`.
- If a snapshot is submitted with `snapshot_time` earlier than or equal to the latest accepted `snapshot_time` for that product version, reject it with 409 Conflict, unless it is the exact duplicate described below.
- Reject exact duplicate snapshots with 409 Conflict when another snapshot already exists for the same `product_name`, `product_version`, `source`, and `snapshot_time`.
- Do not create a new snapshot row or new change rows for rejected duplicates or out-of-order submissions.

Optional bonus: support idempotency_key. If implemented, document whether a repeated key returns the original response or rejects the request.

Change detection rules

Compare the current snapshot with the previous snapshot for the same product version. If there is no previous snapshot, every finding in the current snapshot is new.

Change type	Condition	Record previous/current?	Summary behavior
new	Key exists in current snapshot only.	previous = null, current = current state	Increment new by 1.
resolved	Key exists in previous snapshot only.	previous = previous state, current = null	Increment resolved by 1.
severity_changed	Key exists in both snapshots and severity value changed.	Store previous and current severity, cvss_score, and affected_status.	Increment severity_changed by 1.
status_changed	Key exists in both snapshots and affected_status value changed.	Store previous and current severity, cvss_score, and affected_status.	Increment status_changed by 1.
unchanged	Key exists in both snapshots and neither severity nor affected_status changed.	No individual change row required.	Increment unchanged by 1.

If a finding changes both severity and affected_status in the same comparison, create two change records: one severity_changed record and one status_changed record. The finding is not counted as unchanged.

CVSS score changes should be stored in previous/current fields when another tracked change is recorded. Tracking CVSS-only changes is optional; if implemented, document the behavior.

Database storage

Design the schema yourself, but it must persist at least the following information. Include a setup script or migration that creates the required tables and constraints.

snapshots

- snapshot_id, preferably UUID.
- product_name, product_version, source.
- snapshot_time and created_at.
- previous_snapshot_id, nullable.
- finding_count.
- summary counts: new, resolved, severity_changed, status_changed, unchanged. These can be stored columns or derived from changes plus findings; document your choice.
- Uniqueness constraint for product_name, product_version, source, snapshot_time.

snapshot_findings

- finding_id, snapshot_id.
- vulnerability_id, component_name, component_version, package_url.
- severity, cvss_score, affected_status.
- Uniqueness constraint for snapshot_id, vulnerability_id, component_name, component_version.

snapshot_changes

- change_id, snapshot_id, previous_snapshot_id nullable.
- change_type.
- vulnerability_id, component_name, component_version, package_url optional.
- previous_severity, current_severity.
- previous_cvss_score, current_cvss_score.
- previous_affected_status, current_affected_status.
- created_at.

HTTP API

POST /snapshots

Submit a new snapshot. The service should validate the request, persist the snapshot and findings, compute the diff synchronously, persist change records, and return a summary.

```
201 Created
{
  "snapshot_id": "uuid",
  "product_name": "ExampleDevice",
  "product_version": "1.2.3",
  "previous_snapshot_id": "uuid-or-null",
  "summary": {
    "new": 1,
    "resolved": 0,
    "severity_changed": 0,
    "status_changed": 0,
    "unchanged": 0
  }
}
```

Expected errors: 400 for validation errors or malformed JSON, 409 for exact duplicate or out-of-order snapshot_time, and 500 only for unexpected server failures.

GET /products/{product_name}/versions/{product_version}/snapshots

List snapshots for a product version ordered by snapshot_time descending. Support optional limit and offset. Use sensible defaults and maximums.

```
200 OK
{
  "items": [
    {
      "snapshot_id": "uuid",
      "snapshot_time": "2026-01-01T10:00:00Z",
      "source": "daily-vulnerability-scan",
      "finding_count": 24,
      "created_at": "2026-01-01T10:00:05Z"
    }
  ],
  "limit": 50,
  "offset": 0
}
```

GET /snapshots/{snapshot_id}

Return snapshot metadata and summary. Return 404 if the snapshot does not exist.

```
200 OK
{
  "snapshot_id": "uuid",
  "product_name": "ExampleDevice",
  "product_version": "1.2.3",
  "source": "daily-vulnerability-scan",
  "snapshot_time": "2026-01-01T10:00:00Z",
  "finding_count": 24,
  "previous_snapshot_id": "uuid-or-null",
  "summary": {
    "new": 2,
    "resolved": 1,
    "severity_changed": 3,
    "status_changed": 1,
    "unchanged": 17
  }
}
```

GET /snapshots/{snapshot_id}/changes

List change records for a snapshot. Return 404 if the snapshot does not exist.

- Optional filters: change_type, severity, component_name.
- Optional pagination: limit and offset.
- severity should match either current_severity or previous_severity, whichever is present for the change type.

```
200 OK
{
  "items": [
    {
      "change_type": "new",
      "vulnerability_id": "CVE-2024-12345",
      "component_name": "openssl",
      "component_version": "1.1.1k",
      "package_url": "pkg:generic/openssl@1.1.1k",
      "previous": null,
      "current": {
        "severity": "high",
        "cvss_score": 8.1,
        "affected_status": "affected"
      }
    },
    {
      "change_type": "severity_changed",
      "vulnerability_id": "CVE-2024-99999",
      "component_name": "busybox",
      "component_version": "1.35.0",
      "previous": {
        "severity": "medium",
        "cvss_score": 5.9,
        "affected_status": "affected"
      },
      "current": {
        "severity": "critical",
        "cvss_score": 9.8,
        "affected_status": "affected"
      }
    }
  ],
  "limit": 50,
  "offset": 0
}
```

GET /health

Return a liveness/readiness response. At minimum, return status ok. A stronger implementation also verifies database connectivity.

```
200 OK
{
  "status": "ok"
}
```

Validation and error handling

Return useful, deterministic errors. You may choose the exact error schema, but it should be consistent. Recommended shape:

```
400 Bad Request
{
  "error": {
    "code": "validation_error",
    "message": "Invalid snapshot payload",
    "details": [
```

```
{
  "field": "findings[0].severity", "message": "severity must be one of critical, high, medium, low, none, unknown"}
]
```

Handle at least:

- Malformed JSON.
- Missing or empty product_name.
- Missing or empty product_version.
- Missing or empty source.
- Missing, invalid, or non-UTC snapshot_time.
- Missing or empty findings.
- Missing vulnerability_id.
- Missing component_name or component_version.
- Duplicate matching keys inside a single snapshot.
- Invalid severity.
- Invalid affected_status.
- Invalid cvss_score outside 0.0 to 10.0.
- Unknown snapshot_id on read endpoints.

Required tests

Include automated tests that can be run from the command line. Use pytest or another documented test runner.

1. First snapshot behavior: all findings are new and previous_snapshot_id is null.
2. Second snapshot mixed diff: one new, one resolved, one severity change, one status change, and one unchanged finding.
3. Finding that changes both severity and affected_status creates two change records and is not counted as unchanged.
4. Input validation errors for missing required fields and invalid enum values.
5. Duplicate matching key inside a snapshot is rejected.
6. Exact duplicate snapshot submission is rejected with 409 Conflict.
7. API flow from snapshot submission to retrieving snapshot metadata and changes.
8. Matching-key behavior: same CVE on two component versions is treated as two distinct findings.
9. Pagination and filtering for /changes, at least for change_type.

README requirements

- How to build and run the service locally.
- How to run with Docker, and docker-compose if provided.
- Required environment variables and defaults.
- Database setup or migration instructions.
- How to run tests.
- Matching-key explanation.
- Snapshot ordering, duplicate submission, and idempotency behavior.
- Assumptions and shortcuts.
- What you would improve with more time.

Bonus items

These are not required for a strong submission. Implement them only after the core behavior is correct and tested.

- Idempotency key support.
- Docker Compose with service and database.
- Database integration tests using a real database or test containers.
- Structured JSON logging with request IDs.
- Severity ordering helpers for escalation/de-escalation reporting.
- CVSS-only change tracking or a `cvss_score_changed` change type.
- EPSS or known-exploited fields in the model.
- Async processing and status endpoint for large snapshots.
- Database transaction handling that guarantees snapshot, findings, and changes are committed atomically.
- OpenAPI / Swagger specification.

Example usage

```
curl -X POST http://localhost:8080/snapshots \
  -H "Content-Type: application/json" \
  -d @snapshot-day-1.json

curl -X POST http://localhost:8080/snapshots \
  -H "Content-Type: application/json" \
  -d @snapshot-day-2.json

curl http://localhost:8080/snapshots/<snapshot_id>/changes

curl "http://localhost:8080/snapshots/<snapshot_id>/changes?change_type=severity_changed&limit=20&offset=0"
```